

Computational perspectives on the classical decision problem

Lecture 6 (Prague 12.08.2026)

Bartosz Jan Bednarczyk `bartek@cs.uni.wroc.pl`

TECHNISCHE UNIVERSITÄT WIEN & UNIwersYTET WROCLAWSKI



TECHNISCHE
UNIVERSITÄT
WIEN
Vienna | Austria



Uniwersytet
Wrocławski

Roadmap: An excursion into the linear-equation method

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

1. Add binary-coded counting quantifiers to one-variable logic.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

1. Add **binary-coded counting quantifiers** to one-variable logic.
2. Guess & check method **fails** for NP: models can be **exponentially large**.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

1. Add **binary-coded counting quantifiers** to one-variable logic.
2. Guess & check method **fails** for NP: models can be **exponentially large**.
3. Represent models by the **multiplicities of 1-types**.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

1. Add **binary-coded counting quantifiers** to one-variable logic.
2. Guess & check method **fails** for NP: models can be **exponentially large**.
3. Represent models by the **multiplicities of 1-types**.
4. Translate satisfiability into a **system of linear inequalities**.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

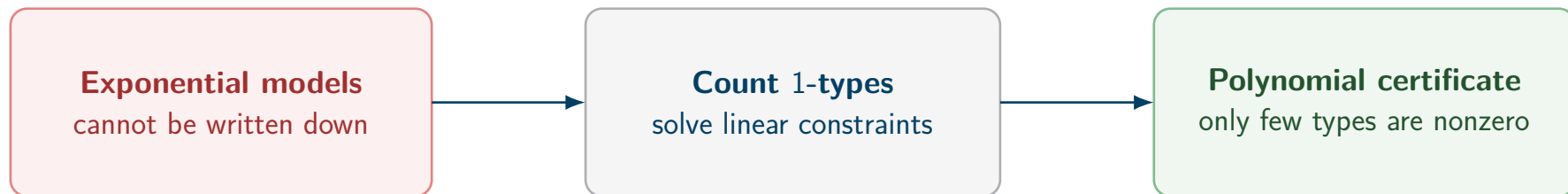
1. Add **binary-coded counting quantifiers** to one-variable logic.
2. Guess & check method **fails** for NP: models can be **exponentially large**.
3. Represent models by the **multiplicities of 1-types**.
4. Translate satisfiability into a **system of linear inequalities**.
5. Compress solutions with a **sparse-support lemma**.

Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

1. Add **binary-coded counting quantifiers** to one-variable logic.
2. Guess & check method **fails** for NP: models can be **exponentially large**.
3. Represent models by the **multiplicities of 1-types**.
4. Translate satisfiability into a **system of linear inequalities**.
5. Compress solutions with a **sparse-support lemma**.

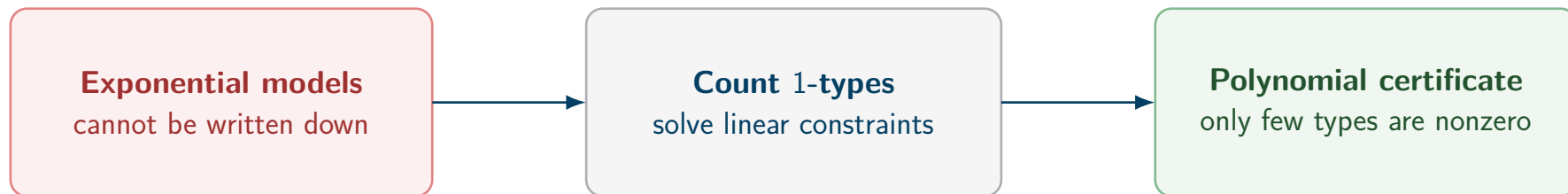


Roadmap: An excursion into the linear-equation method

C^1 = one-variable logic with counting quantifiers $\exists^{\leq n}x.\varphi + \exists^{\geq n}x.\varphi$.

Goal: $\text{SAT}(C^1)$ is NP-complete.

1. Add **binary-coded counting quantifiers** to one-variable logic.
2. Guess & check method **fails** for NP: models can be **exponentially large**.
3. Represent models by the **multiplicities of 1-types**.
4. Translate satisfiability into a **system of linear inequalities**.
5. Compress solutions with a **sparse-support lemma**.



Feel free to ask questions and interrupt me!

Please don't be shy! If needed, send me an email (bartek@cs.uni.wroc.pl) or approach me before/after the lecture!

If you would like to do a research project on related topics, let me know!

Icon downloaded from icon-icons.com by ©VisualPharm (under CC BY 4.0). No changes have been made.

Definition of C^1 and some remarks

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones.

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^{=C}$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.



Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^{=C}$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.

Compare $\exists x_1 \dots \exists x_{1000000} \bigwedge_{1 \leq i < j \leq 1000000} x_i \neq x_j$ and $\exists^{\geq 1000000} x \top$.



Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.

Compare $\exists x_1 \dots \exists x_{1000000} \bigwedge_{1 \leq i < j \leq 1000000} x_i \neq x_j$ and $\exists^{\geq 1000000} x \top$.



Dangerous consequence: Guess and verify method no longer works if we aim at NP.

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.

Compare $\exists x_1 \dots \exists x_{1000000} \bigwedge_{1 \leq i < j \leq 1000000} x_i \neq x_j$ and $\exists^{\geq 1000000} x \top$.

Dangerous consequence: Guess and verify method no longer works if we aim at NP.

$$\varphi_n := \exists^{\geq 2^n} x \top$$



Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.

Compare $\exists x_1 \dots \exists x_{1000000} \bigwedge_{1 \leq i < j \leq 1000000} x_i \neq x_j$ and $\exists^{\geq 1000000} x \top$.



Dangerous consequence: Guess and verify method no longer works if we aim at NP.

$$\varphi_n := \exists^{\geq 2^n} x \top$$

- Size-wise $|\varphi_n| \in O(n)$

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.

Compare $\exists x_1 \dots \exists x_{1000000} \bigwedge_{1 \leq i < j \leq 1000000} x_i \neq x_j$ and $\exists^{\geq 1000000} x \top$.



Dangerous consequence: Guess and verify method no longer works if we aim at NP.

$$\varphi_n := \exists^{\geq 2^n} x \top$$

- Size-wise $|\varphi_n| \in O(n)$
- Every model of φ_n contains $\geq 2^n$ elements.

Definition of C^1 and some remarks

Definition (C^1)

FO^1 enriched with $\exists^{\geq C}$, $\exists^{\leq C}$ and $\exists^=C$ for $C \in \mathbb{N}$.

$$\mathfrak{A} \models \exists^{\geq C} x \psi(x) \iff |\{a \in A : \mathfrak{A} \models \psi[a]\}| \geq C.$$

Remark: Counting quantifiers generalise the usual ones. $\exists x \psi \equiv \exists^{\geq 1} x \psi$ and $\forall x \psi \equiv \exists^{\leq 0} x \neg \psi$.

Coding matters! C is written in binary $\approx$ only $\lfloor \log_2 C \rfloor + 1$ bits.

Compare $\exists x_1 \dots \exists x_{1000000} \bigwedge_{1 \leq i < j \leq 1000000} x_i \neq x_j$ and $\exists^{\geq 1000000} x \top$.



Dangerous consequence: Guess and verify method no longer works if we aim at NP.

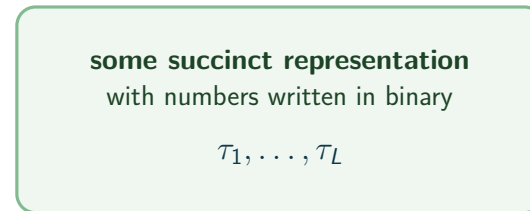
$$\varphi_n := \exists^{\geq 2^n} x \top$$

• Size-wise $|\varphi_n| \in O(n)$

• Every model of φ_n contains $\geq 2^n$ elements.



compress
→



Running examples of C^1 -formulae

Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

Running examples of C^1 -formulae

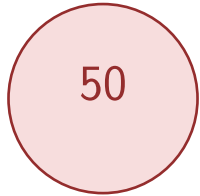
Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$

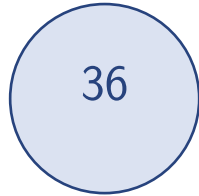
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

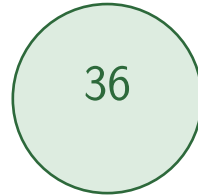
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



French



German



Spanish

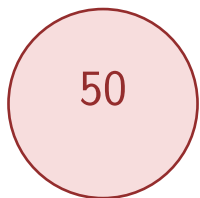


silent

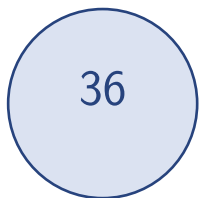
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

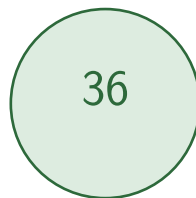
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



French



German



Spanish



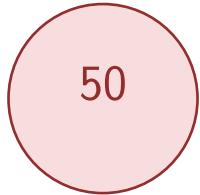
silent

Is ψ_0 satisfiable?

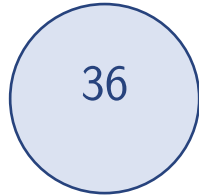
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

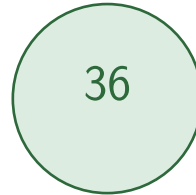
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



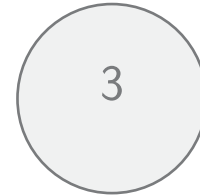
French



German



Spanish



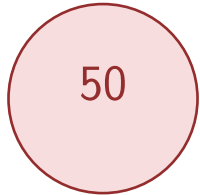
silent

Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

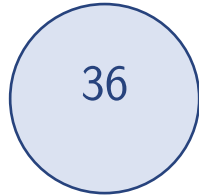
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

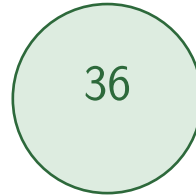
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



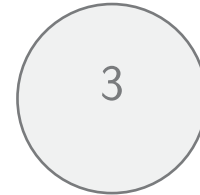
French



German



Spanish



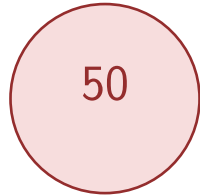
silent

Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

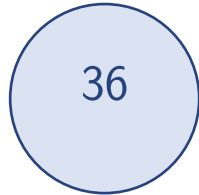
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

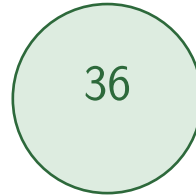
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



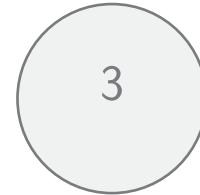
French



German



Spanish



silent

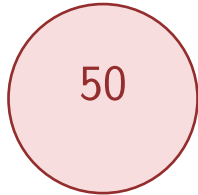
Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

How about $\psi_1 := \psi_0 \wedge \forall x (Fr(x) \vee Ge(x) \vee Sp(x))$?

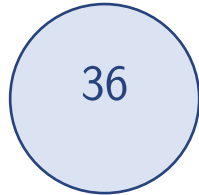
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

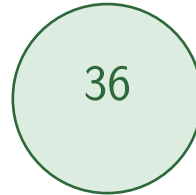
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



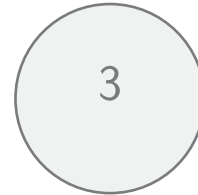
French



German



Spanish



silent

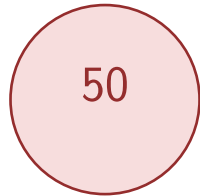
Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

How about $\psi_1 := \psi_0 \wedge \forall x (Fr(x) \vee Ge(x) \vee Sp(x))$? **No!**

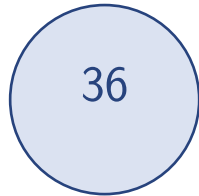
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

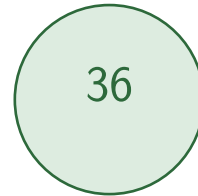
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



French



German



Spanish



silent

Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

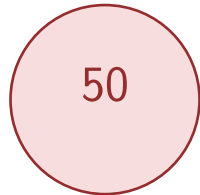
How about $\psi_1 := \psi_0 \wedge \forall x (Fr(x) \vee Ge(x) \vee Sp(x))$? **No!**

- Now every element belongs to the union of the three language sets, hence

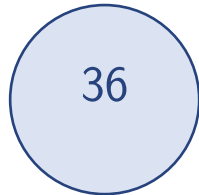
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

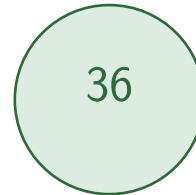
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



French



German



Spanish



silent

Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

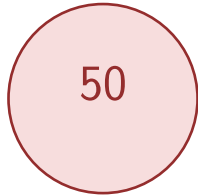
How about $\psi_1 := \psi_0 \wedge \forall x (Fr(x) \vee Ge(x) \vee Sp(x))$? **No!**

- Now every element belongs to the union of the three language sets, hence
- $|A| \leq |Fr^{\mathfrak{A}}| + |Ge^{\mathfrak{A}}| + |Sp^{\mathfrak{A}}| = 50 + 36 + 36 = 122$

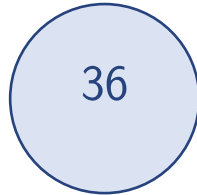
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

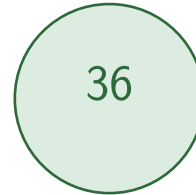
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



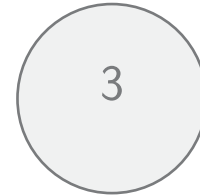
French



German



Spanish



silent

Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

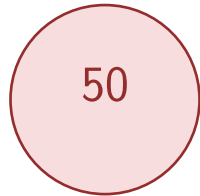
How about $\psi_1 := \psi_0 \wedge \forall x (Fr(x) \vee Ge(x) \vee Sp(x))$? **No!**

- Now every element belongs to the union of the three language sets, hence
- $|A| \leq |Fr^{\mathfrak{A}}| + |Ge^{\mathfrak{A}}| + |Sp^{\mathfrak{A}}| = 50 + 36 + 36 = 122$ (for the supposed model $\mathfrak{A}$ of ψ_1).

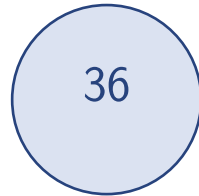
Running examples of C^1 -formulae

Let $Fr(x)$, $Ge(x)$, $Sp(x)$ mean that a person speaks French, German or Spanish.

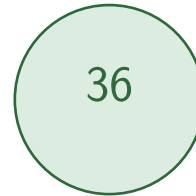
$$\psi_0 := \exists^{\geq 125} x \top \wedge \exists^{=50} x Fr(x) \wedge \exists^{=36} x Ge(x) \wedge \exists^{=36} x Sp(x)$$



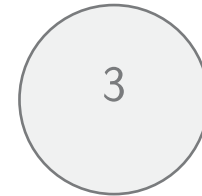
French



German



Spanish



silent

Is ψ_0 satisfiable? **Yes!** (because $50 + 36 + 36 + 3 = 125$).

How about $\psi_1 := \psi_0 \wedge \forall x (Fr(x) \vee Ge(x) \vee Sp(x))$? **No!**

- Now every element belongs to the union of the three language sets, hence
- $|A| \leq |Fr^{\mathfrak{A}}| + |Ge^{\mathfrak{A}}| + |Sp^{\mathfrak{A}}| = 50 + 36 + 36 = 122$ (for the supposed model $\mathfrak{A}$ of ψ_1).

required by ψ_0
 $|A| \geq 125$

← contradiction →

forced by coverage
 $|A| \leq 122$

A simplification: Normal form for C^1

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i C_i} x \theta_i(x),$$

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i} C_i x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i} c_i x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i} c_i x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

1. For simplicity: assume that φ does not have non-unary predicates or constants.

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i} C_i x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

1. For simplicity: assume that φ does not have non-unary predicates or constants.
2. Choose an innermost closed counting subformula

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i C_i} x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

1. For simplicity: assume that φ does not have non-unary predicates or constants.
2. Choose an innermost closed counting subformula

$$\eta := \exists^{\bowtie C} x \chi(x), \quad \chi(x) \text{ quantifier-free.}$$

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i C_i} x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

1. For simplicity: assume that φ does not have non-unary predicates or constants.
2. Choose an innermost closed counting subformula

$$\eta := \exists^{\bowtie C} x \chi(x), \quad \chi(x) \text{ quantifier-free.}$$

3. Guess whether η is true or false in the intended model.

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i C_i} x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

1. For simplicity: assume that φ does not have non-unary predicates or constants.
2. Choose an innermost closed counting subformula
$$\eta := \exists^{\bowtie C} x \chi(x), \quad \chi(x) \text{ quantifier-free.}$$
3. Guess whether η is true or false in the intended model.
4. Record one top-level counting constraint forcing that guess, then simplify the outcome.

A simplification: Normal form for C^1

Lemma (Nondeterministic normal form)

There is an NP procedure that given a C^1 sentence φ outputs a β an equisatisfiable ψ of the form:

$$\psi = \bigwedge_{i=1}^m \exists^{\bowtie_i C_i} x \theta_i(x),$$

where $m \leq |\varphi|$, $\bowtie_i \in \{\leq, \geq, =\}$, and every $\theta_i(x)$ is quantifier-free and contains only monadic predicates.

Proof

1. For simplicity: assume that φ does not have non-unary predicates or constants.
2. Choose an innermost closed counting subformula
$$\eta := \exists^{\bowtie C} x \chi(x), \quad \chi(x) \text{ quantifier-free.}$$
3. Guess whether η is true or false in the intended model.
4. Record one top-level counting constraint forcing that guess, then simplify the outcome.

$$\varphi \rightsquigarrow \text{either } \varphi[\eta/\top] \wedge \eta \text{ or } \varphi[\eta/\perp] \wedge \text{simplify}(\neg\eta)$$

1-types strike back!

1-types strike back!

Definition (1-type)

A complete truth assignment to the unary predicates. Over $\{Fr, Ge, Sp\}$ there are $2^3 = 8$ types.

1-types strike back!

Definition (1-type)

A complete truth assignment to the unary predicates. Over $\{Fr, Ge, Sp\}$ there are $2^3 = 8$ types.



1-types strike back!

Definition (1-type)

A complete truth assignment to the unary predicates. Over $\{Fr, Ge, Sp\}$ there are $2^3 = 8$ types.



1-types are complete descriptions: the truth of a quantifier-free $\theta(x)$ is determined by a 1-type.

A model becomes a characteristic vector

A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$



A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$

We call $\alpha(\mathfrak{A})$ the **characteristic vector** of $\mathfrak{A}$.



A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$

We call $\alpha(\mathfrak{A})$ the **characteristic vector** of $\mathfrak{A}$.



Up to isomorphism, $\alpha(\mathfrak{A})$ contains everything a C^1 sentence can see.

A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$

We call $\alpha(\mathfrak{A})$ the **characteristic vector** of $\mathfrak{A}$.



Up to isomorphism, $\alpha(\mathfrak{A})$ contains everything a C^1 sentence can see.

A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$

We call $\alpha(\mathfrak{A})$ the **characteristic vector** of $\mathfrak{A}$.



Up to isomorphism, $\alpha(\mathfrak{A})$ contains everything a C^1 sentence can see.

For the language example, one coordinate is attached to each type:

A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$

We call $\alpha(\mathfrak{A})$ the **characteristic vector** of $\mathfrak{A}$.



Up to isomorphism, $\alpha(\mathfrak{A})$ contains everything a C^1 sentence can see.

For the language example, one coordinate is attached to each type:

$$\alpha = (x_{\emptyset}, x_{Fr}, x_{Ge}, x_{Sp}, x_{FrGe}, x_{FrSp}, x_{GeSp}, x_{FrGeSp}).$$

A model becomes a characteristic vector

Enumerate the $L = 2^\ell$ types as $\pi_1, \dots, \pi_L$. For a structure $\mathfrak{A}$ we define

$$\alpha(\mathfrak{A}) = (\alpha_1, \dots, \alpha_L), \quad \alpha_j := |\{a \in A : \text{tp}^{\mathfrak{A}}(a) = \pi_j\}|.$$

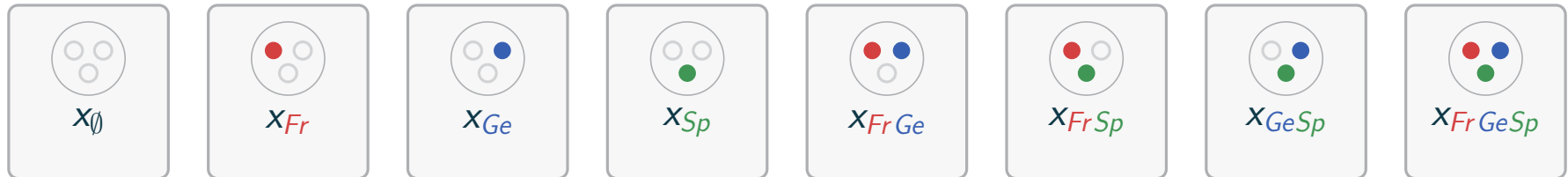
We call $\alpha(\mathfrak{A})$ the **characteristic vector** of $\mathfrak{A}$.



Up to isomorphism, $\alpha(\mathfrak{A})$ contains everything a C^1 sentence can see.

For the language example, one coordinate is attached to each type:

$$\alpha = (x_{\emptyset}, x_{Fr}, x_{Ge}, x_{Sp}, x_{FrGe}, x_{FrSp}, x_{GeSp}, x_{FrGeSp}).$$



From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the coefficient $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the **coefficient** $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists \bowtie_i^{C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the **coefficient** $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists \bowtie_i^{C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the **coefficient** $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists \bowtie_i C_i x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

Examples over $\{Fr, Ge, Sp\}$:

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the **coefficient** $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists^{\bowtie_i C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

Examples over $\{Fr, Ge, Sp\}$:

- $\exists^{=125} x \top$ becomes $x_\emptyset + x_{Fr} + x_{Ge} + x_{Sp} + x_{FrGe} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 125$.

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the coefficient $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists^{\bowtie_i C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

Examples over $\{Fr, Ge, Sp\}$:

- $\exists^{=125} x \top$ becomes $x_\emptyset + x_{Fr} + x_{Ge} + x_{Sp} + x_{FrGe} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 125$.
- $\exists^{=50} x Fr(x)$ becomes $x_{Fr} + x_{FrGe} + x_{FrSp} + x_{FrGeSp} = 50$.

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the coefficient $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists^{\bowtie_i C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

Examples over $\{Fr, Ge, Sp\}$:

- $\exists^{=125} x \top$ becomes $x_\emptyset + x_{Fr} + x_{Ge} + x_{Sp} + x_{FrGe} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 125$.
- $\exists^{=50} x Fr(x)$ becomes $x_{Fr} + x_{FrGe} + x_{FrSp} + x_{FrGeSp} = 50$.
- $\exists^{=36} x Ge(x)$ becomes $x_{Ge} + x_{FrGe} + x_{GeSp} + x_{FrGeSp} = 36$.

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the **coefficient** $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists^{\bowtie_i C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

The coefficients are Boolean; the unknowns are nonnegative integers.

Examples over $\{Fr, Ge, Sp\}$:

- $\exists^{=125} x \top$ becomes $x_\emptyset + x_{Fr} + x_{Ge} + x_{Sp} + x_{FrGe} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 125$.
- $\exists^{=50} x Fr(x)$ becomes $x_{Fr} + x_{FrGe} + x_{FrSp} + x_{FrGeSp} = 50$.
- $\exists^{=36} x Ge(x)$ becomes $x_{Ge} + x_{FrGe} + x_{GeSp} + x_{FrGeSp} = 36$.
- $\exists^{=36} x Sp(x)$ becomes $x_{Sp} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 36$.

From C^1 to Linear Algebra: Each counting conjunct becomes a linear constraint

For the i -th subformula θ_i and a 1-type π_j , define the **coefficient** $a_{i,j} := \begin{cases} 1 & \text{if } \pi_j \models \theta_i, \\ 0 & \text{otherwise.} \end{cases}$

Lemma (Translation)

$\exists^{\bowtie_i C_i} x \theta_i(x)$ is true exactly when $a_{i,1}x_1 + \dots + a_{i,L}x_L \bowtie_i C_i$.

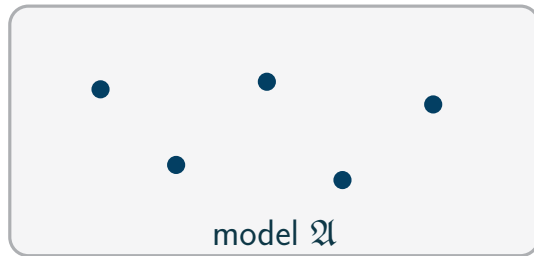
The coefficients are Boolean; the unknowns are nonnegative integers.

Examples over $\{Fr, Ge, Sp\}$:

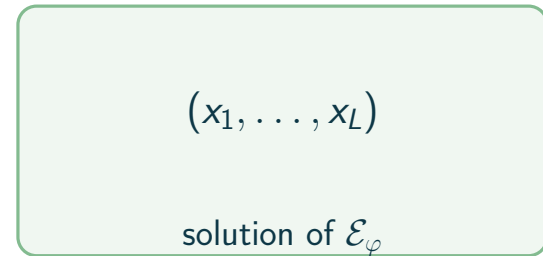
- $\exists^{=125} x \top$ becomes $x_\emptyset + x_{Fr} + x_{Ge} + x_{Sp} + x_{FrGe} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 125$.
- $\exists^{=50} x Fr(x)$ becomes $x_{Fr} + x_{FrGe} + x_{FrSp} + x_{FrGeSp} = 50$.
- $\exists^{=36} x Ge(x)$ becomes $x_{Ge} + x_{FrGe} + x_{GeSp} + x_{FrGeSp} = 36$.
- $\exists^{=36} x Sp(x)$ becomes $x_{Sp} + x_{FrSp} + x_{GeSp} + x_{FrGeSp} = 36$.
- $\forall x (Fr(x) \vee Ge(x) \vee Sp(x)) \equiv \exists^{=0} x (\neg Fr(x) \wedge \neg Ge(x) \wedge \neg Sp(x))$ becomes $x_\emptyset = 0$.

Equivalence between models and linear inequalities over $\mathbb{N}$

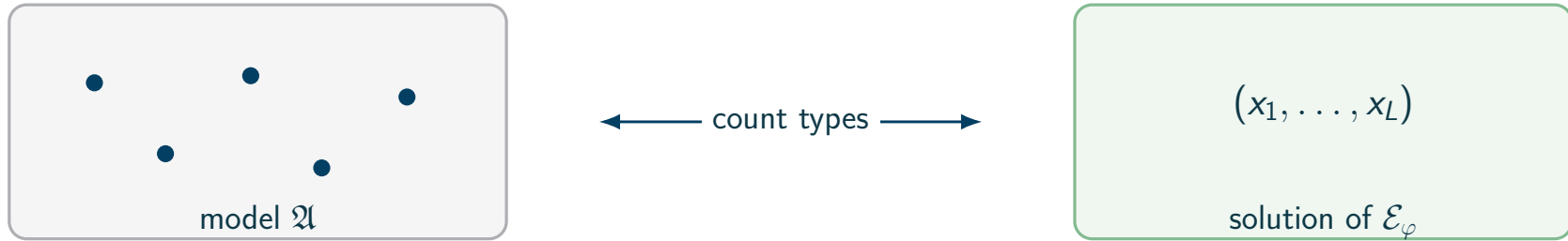
Equivalence between models and linear inequalities over $\mathbb{N}$



← count types →



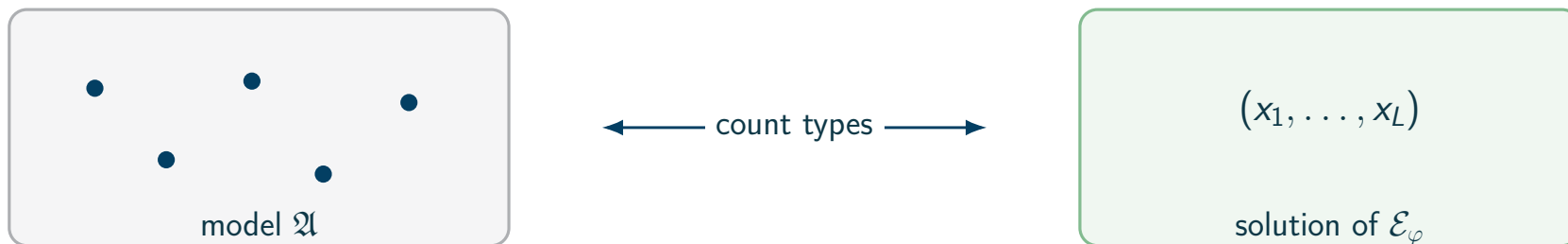
Equivalence between models and linear inequalities over $\mathbb{N}$



Lemma (Correspondence)

For φ in normal form, let $\mathcal{E}_\varphi$ contain the constraints $\sum_{j=1}^L a_{i,j}x_j \bowtie_i C_i \quad (1 \leq i \leq m), \quad x_j \in \mathbb{N}$

Equivalence between models and linear inequalities over $\mathbb{N}$

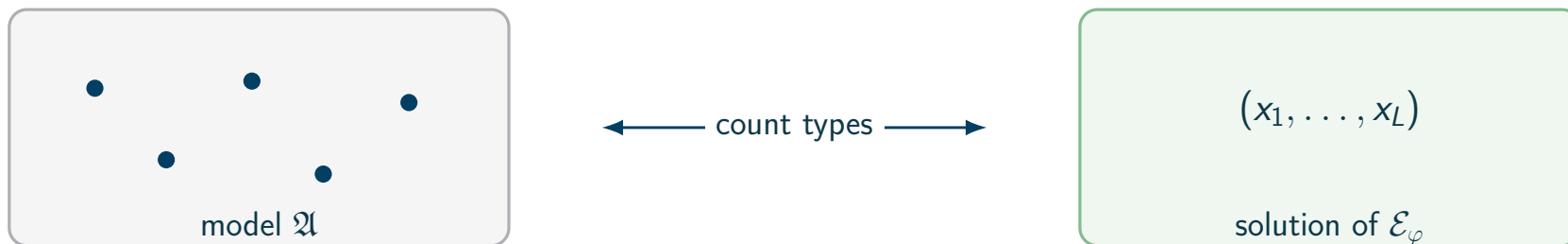


Lemma (Correspondence)

For φ in normal form, let $\mathcal{E}_\varphi$ contain the constraints $\sum_{j=1}^L a_{i,j}x_j \bowtie_i C_i \quad (1 \leq i \leq m), \quad x_j \in \mathbb{N}$

$$\mathfrak{A} \models \varphi \iff \alpha(\mathfrak{A}) \text{ is a solution to } \mathcal{E}_\varphi.$$

Equivalence between models and linear inequalities over $\mathbb{N}$



Lemma (Correspondence)

For φ in normal form, let $\mathcal{E}_\varphi$ contain the constraints $\sum_{j=1}^L a_{i,j}x_j \bowtie_i C_i \quad (1 \leq i \leq m), \quad x_j \in \mathbb{N}$

$$\mathfrak{A} \models \varphi \iff \alpha(\mathfrak{A}) \text{ is a solution to } \mathcal{E}_\varphi.$$

Model $\Rightarrow$ vector

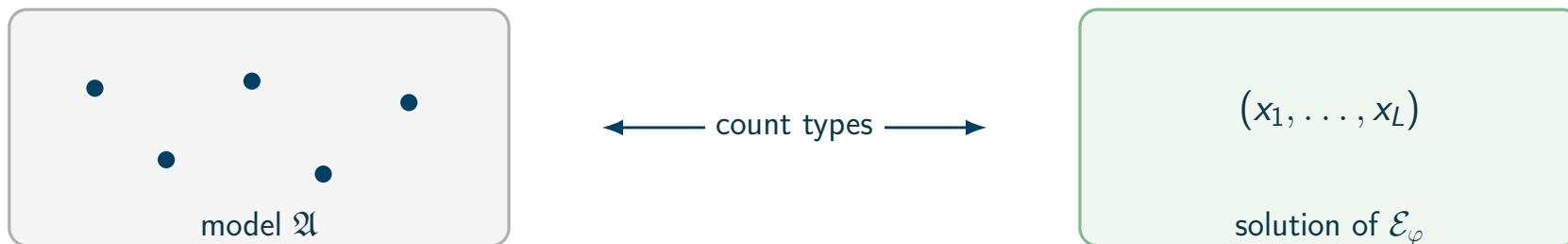
Set x_j to the number of elements of type π_j .

Since the type classes partition the domain

and every type decides θ_i , we have

$$|\theta_i^{\mathfrak{A}}| = \sum_j a_{i,j}x_j.$$

Equivalence between models and linear inequalities over $\mathbb{N}$



Lemma (Correspondence)

For φ in normal form, let $\mathcal{E}_\varphi$ contain the constraints $\sum_{j=1}^L a_{i,j}x_j \bowtie_i C_i \quad (1 \leq i \leq m), \quad x_j \in \mathbb{N}$

$$\mathfrak{A} \models \varphi \iff \alpha(\mathfrak{A}) \text{ is a solution to } \mathcal{E}_\varphi.$$

Model $\Rightarrow$ vector

Set x_j to the number of elements of type π_j .
 Since the type classes partition the domain
 and every type decides θ_i , we have

$$|\theta_i^{\mathfrak{A}}| = \sum_j a_{i,j}x_j.$$

Vector $\Rightarrow$ model

For each j , create a disjoint set A_j of size x_j
 and assign every element of A_j the type π_j .
 Then the same identity shows that all
 counting constraints hold.

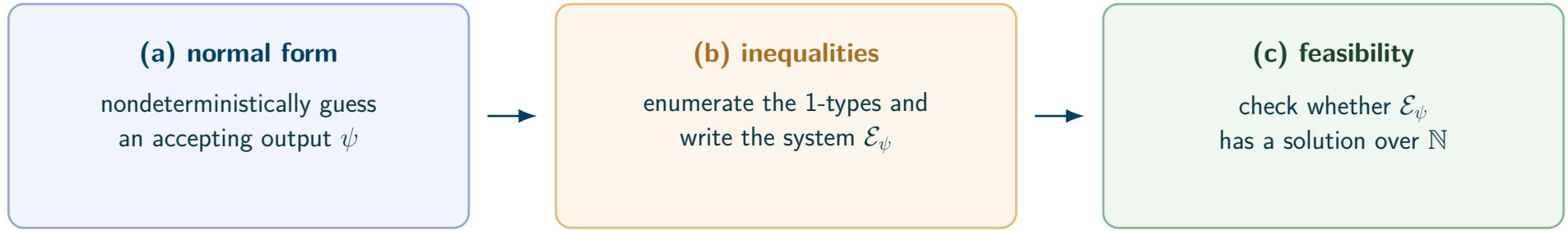
The decision procedure for C^1

The decision procedure for C^1

We have developed the following decision procedure for an input $\varphi \in C^1$:

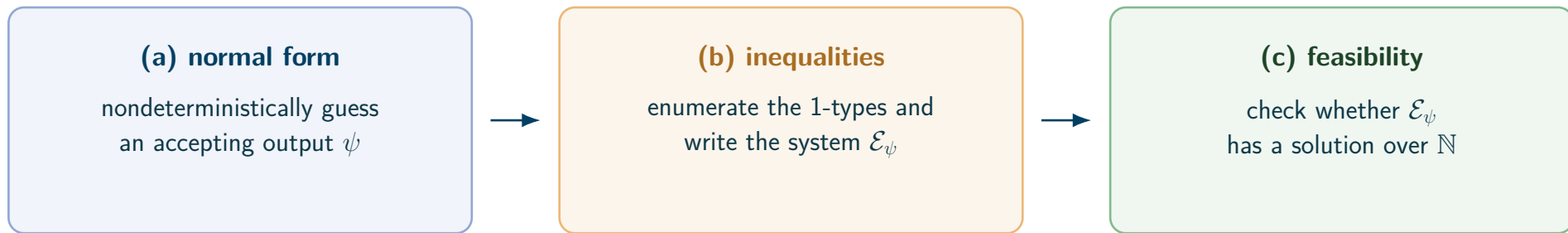
The decision procedure for C^1

We have developed the following decision procedure for an input $\varphi \in C^1$:



The decision procedure for C^1

We have developed the following decision procedure for an input $\varphi \in C^1$:

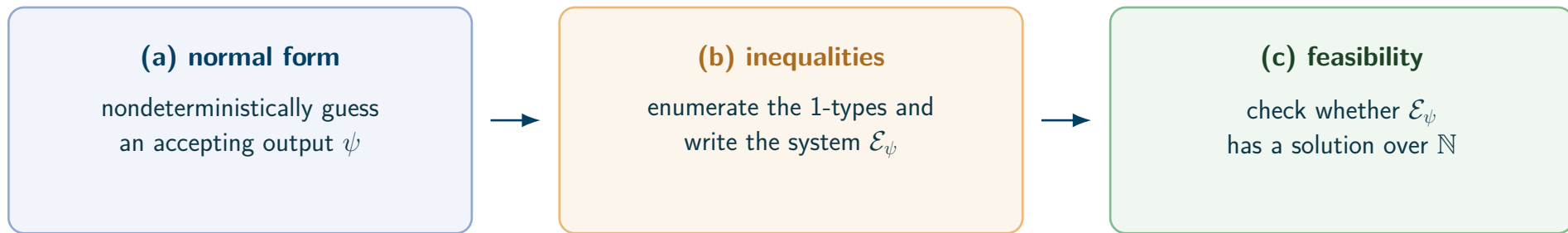


Theorem (Borosh–Treybig 1976)

Integer programming — does a system of linear (in)equalities have a solution over $\mathbb{N}$? — is in NP.

The decision procedure for C^1

We have developed the following decision procedure for an input $\varphi \in C^1$:

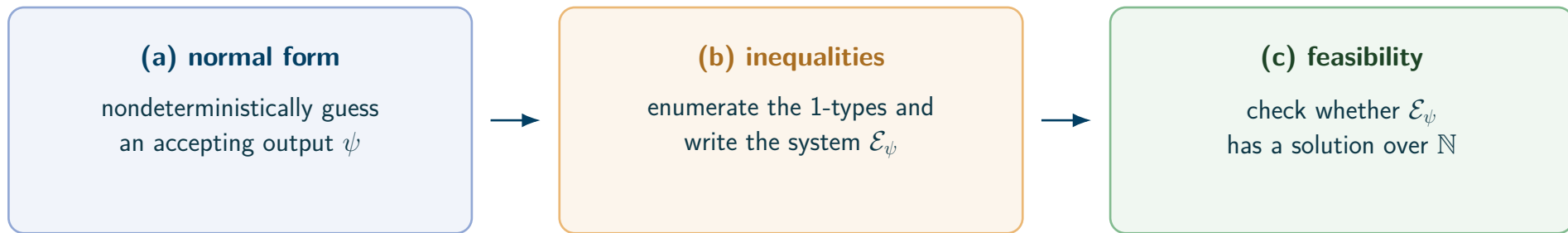


Theorem (Borosh–Treybig 1976)

Integer programming — does a system of linear (in)equalities have a solution over $\mathbb{N}$? — is in NP.

The decision procedure for C^1

We have developed the following decision procedure for an input $\varphi \in C^1$:



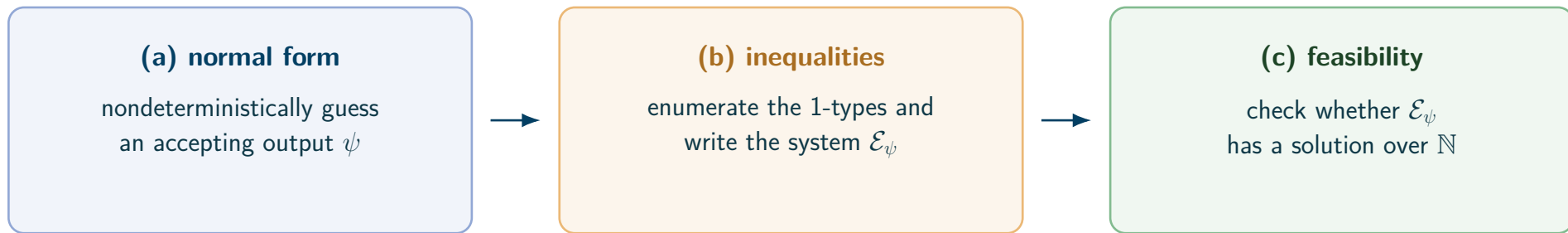
Theorem (Borosh–Treybig 1976)

Integer programming — does a system of linear (in)equalities have a solution over $\mathbb{N}$? — is in NP.

Is this the end of the story?

The decision procedure for C^1

We have developed the following decision procedure for an input $\varphi \in C^1$:



Theorem (Borosh–Treybig 1976)

Integer programming — does a system of linear (in)equalities have a solution over $\mathbb{N}$? — is in NP.

Is this the end of the story?

The explicit system has $L = 2^\ell$ variables, so $|\mathcal{E}_\psi|$ may be exponential in $|\varphi|$.
NP in the system size yields only NExpTime in the formula size: we are not at NP yet.



Can this be fixed?

Can this be fixed?

too much data

2^ℓ coordinates

$(x_1, x_2, \dots, x_L)$

— keep only
nonzero types →

desired certificate

polynomial support

$(\pi_{j_1}, x_{j_1}), \dots, (\pi_{j_k}, x_{j_k})$

Can this be fixed?

too much data

2^ℓ coordinates

$(x_1, x_2, \dots, x_L)$

— keep only
nonzero types →

desired certificate

polynomial support

$(\pi_{j_1}, x_{j_1}), \dots, (\pi_{j_k}, x_{j_k})$

Question

Can every feasible $\mathcal{E}_\varphi$ be solved with **only polynomially many** nonzero variables?



Can this be fixed?

too much data

2^ℓ coordinates

$(x_1, x_2, \dots, x_L)$

— keep only
nonzero types →

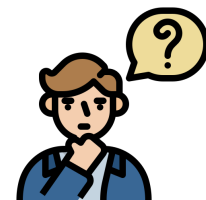
desired certificate

polynomial support

$(\pi_{j_1}, x_{j_1}), \dots, (\pi_{j_k}, x_{j_k})$

Question

Can every feasible $\mathcal{E}_\varphi$ be solved with **only polynomially many** nonzero variables?



Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

Can this be fixed?

too much data

2^ℓ coordinates

$(x_1, x_2, \dots, x_L)$

— keep only
nonzero types →

desired certificate

polynomial support

$(\pi_{j_1}, x_{j_1}), \dots, (\pi_{j_k}, x_{j_k})$

Question

Can every feasible $\mathcal{E}_\varphi$ be solved with **only polynomially many** nonzero variables?



Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

The improved nondeterministic algorithm (this time in NP):

Can this be fixed?

too much data

2^ℓ coordinates

$(x_1, x_2, \dots, x_L)$

— keep only
nonzero types →

desired certificate

polynomial support

$(\pi_{j_1}, x_{j_1}), \dots, (\pi_{j_k}, x_{j_k})$

Question

Can every feasible $\mathcal{E}_\varphi$ be solved with **only polynomially many** nonzero variables?



Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

The improved nondeterministic algorithm (this time in NP):

1. Guess polynomially many variables that will have non-zero values assigned.

Can this be fixed?

too much data

2^ℓ coordinates

$(x_1, x_2, \dots, x_L)$

— keep only
nonzero types →

desired certificate

polynomial support

$(\pi_{j_1}, x_{j_1}), \dots, (\pi_{j_k}, x_{j_k})$

Question

Can every feasible $\mathcal{E}_\varphi$ be solved with **only polynomially many** nonzero variables?



Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

The improved nondeterministic algorithm (this time in NP):

1. Guess polynomially many variables that will have non-zero values assigned.
2. Write the system of inequalities restricted to those variables and solve it.

Magic lemma — proof

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$,

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$, so there are at most $(L + 1)^m$ distinct vectors $\mathbf{v}_I$.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$, so there are at most $(L + 1)^m$ distinct vectors $\mathbf{v}_I$.

If $k > m \log(L + 1)$ then $2^k > (L + 1)^m$: two subsets collide, $\mathbf{v}_I = \mathbf{v}_{I'}$.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$, so there are at most $(L + 1)^m$ distinct vectors $\mathbf{v}_I$.

If $k > m \log(L + 1)$ then $2^k > (L + 1)^m$: two subsets collide, $\mathbf{v}_I = \mathbf{v}_{I'}$.

Set $J = I \setminus I'$, $J' = I' \setminus I$ (disjoint, $\mathbf{v}_J = \mathbf{v}_{J'}$, wlog $J \neq \emptyset$). Adjust

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$, so there are at most $(L + 1)^m$ distinct vectors $\mathbf{v}_I$.

If $k > m \log(L + 1)$ then $2^k > (L + 1)^m$: two subsets collide, $\mathbf{v}_I = \mathbf{v}_{I'}$.

Set $J = I \setminus I'$, $J' = I' \setminus I$ (disjoint, $\mathbf{v}_J = \mathbf{v}_{J'}$, wlog $J \neq \emptyset$). Adjust

$$b'_j = b_j - 1 \ (j \in J), \quad b'_j = b_j + 1 \ (j \in J'), \quad b'_j = b_j \text{ else.}$$

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$, so there are at most $(L + 1)^m$ distinct vectors $\mathbf{v}_I$.

If $k > m \log(L + 1)$ then $2^k > (L + 1)^m$: two subsets collide, $\mathbf{v}_I = \mathbf{v}_{I'}$.

Set $J = I \setminus I'$, $J' = I' \setminus I$ (disjoint, $\mathbf{v}_J = \mathbf{v}_{J'}$, wlog $J \neq \emptyset$). Adjust

$$b'_j = b_j - 1 \ (j \in J), \quad b'_j = b_j + 1 \ (j \in J'), \quad b'_j = b_j \text{ else.}$$

Then $\mathbf{b}' \geq 0$ and $A\mathbf{b}' = \mathbf{c}$; repeating kills a nonzero — contradiction.

Magic lemma — proof

Lemma (Sparse-solution lemma)

If a system $\mathcal{E}$ of m Boolean equations in L variables has a solution over $\mathbb{N}$, then it has one with at most $m \log_2(L + 1)$ nonzero entries.

For simplicity: we work on Boolean equations, not inequalities.

Write $\mathcal{E}$ as $A\mathbf{x} = \mathbf{c}$, $A \in \{0, 1\}^{m \times L}$. Take a solution $\mathbf{b} \geq 0$ with the fewest nonzeros, say k ; assume $b_1, \dots, b_k > 0$ and $k > 0$.

For $I \subseteq \{1, \dots, k\}$ let $\mathbf{v}_I = \sum_{j \in I} (\text{column } j)$. Each entry is in $\{0, \dots, L\}$, so there are at most $(L + 1)^m$ distinct vectors $\mathbf{v}_I$.

If $k > m \log(L + 1)$ then $2^k > (L + 1)^m$: two subsets collide, $\mathbf{v}_I = \mathbf{v}_{I'}$.

Set $J = I \setminus I'$, $J' = I' \setminus I$ (disjoint, $\mathbf{v}_J = \mathbf{v}_{J'}$, wlog $J \neq \emptyset$). Adjust

$$b'_j = b_j - 1 \ (j \in J), \quad b'_j = b_j + 1 \ (j \in J'), \quad b'_j = b_j \text{ else.}$$

Then $\mathbf{b}' \geq 0$ and $A\mathbf{b}' = \mathbf{c}$; repeating kills a nonzero — contradiction.

Take-away messages

Take-away messages

1. C^1 is one-variable logic plus binary-coded counting quantifiers.

Take-away messages

1. C^1 is one-variable logic plus binary-coded counting quantifiers.
2. Minimal models may be exponential: explicit model guessing is impossible in NP.

Take-away messages

1. C^1 is one-variable logic plus binary-coded counting quantifiers.
2. Minimal models may be exponential: explicit model guessing is impossible in NP.
3. A model is represented by characteristic vectors (multiplicities of its 1-types).

Take-away messages

1. C^1 is one-variable logic plus **binary-coded counting quantifiers**.
2. Minimal models **may be exponential**: explicit model guessing is impossible in NP.
3. A model is represented by **characteristic vectors** (multiplicities of its 1-types).
4. The model constraints form a **Boolean-coefficient integer system** $\mathcal{E}_\varphi$.

Take-away messages

1. C^1 is one-variable logic plus **binary-coded counting quantifiers**.
2. Minimal models **may be exponential**: explicit model guessing is impossible in NP.
3. A model is represented by **characteristic vectors** (multiplicities of its 1-types).
4. The model constraints form a **Boolean-coefficient integer system** $\mathcal{E}_\varphi$.
5. Every feasible system has a **sparse solution** with at most $m \log_2(L + 1)$ nonzeros.

Take-away messages

1. C^1 is one-variable logic plus **binary-coded counting quantifiers**.
2. Minimal models **may be exponential**: explicit model guessing is impossible in NP.
3. A model is represented by **characteristic vectors** (multiplicities of its 1-types).
4. The model constraints form a **Boolean-coefficient integer system** $\mathcal{E}_\varphi$.
5. Every feasible system has a **sparse solution** with at most $m \log_2(L + 1)$ nonzeros.
6. A similar approach works for **more expressive forms of counting**.

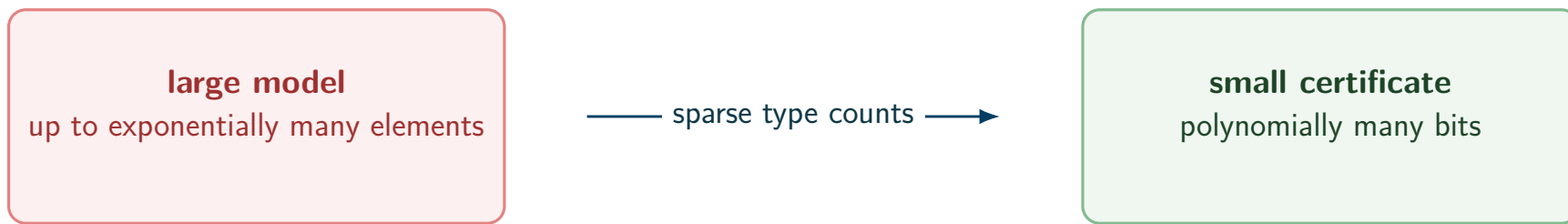
Take-away messages

1. C^1 is one-variable logic plus **binary-coded counting quantifiers**.
2. Minimal models **may be exponential**: explicit model guessing is impossible in NP.
3. A model is represented by **characteristic vectors** (multiplicities of its 1-types).
4. The model constraints form a **Boolean-coefficient integer system** $\mathcal{E}_\varphi$.
5. Every feasible system has a **sparse solution** with at most $m \log_2(L + 1)$ nonzeros.
6. A similar approach works for **more expressive forms of counting**.



Take-away messages

1. C^1 is one-variable logic plus **binary-coded counting quantifiers**.
2. Minimal models **may be exponential**: explicit model guessing is impossible in NP.
3. A model is represented by **characteristic vectors** (multiplicities of its 1-types).
4. The model constraints form a **Boolean-coefficient integer system** $\mathcal{E}_\varphi$.
5. Every feasible system has a **sparse solution** with at most $m \log_2(L + 1)$ nonzeros.
6. A similar approach works for **more expressive forms of counting**.



$\text{SAT}(C^1)$ is NP-complete.